

Data Structure: 자료구조 구현을 위한 C 프로그래밍 기법

Prof. Jungmin Kwon
Spring, 2026

■ 학습목표

- 자료구조를 C 프로그램으로 구현하기 위해 필요한 프로그래밍 기법을 알아본다.
- 배열 자료형을 이해하고 배열의 구현 방법을 알아본다.
- 포인터의 의미를 이해하고 구현 방법을 알아본다.
- 구조체 자료형을 이해하고 구현 방법을 알아본다.
- 재귀호출의 의미를 이해하고 구현 방법을 알아본다.

■ 내용

01 배열

02 포인터

03 구조체

04 재귀호출

1. 배열 : 개념

■ 배열 array

- 같은 자료형을 가진 자료들을 나열하여 메모리에 연속으로 저장하여 만든 자료들의 그룹
- 인덱스 index
 - 배열의 요소를 간단히 구별하기 위해 사용하는 번호
 - C에서 인덱스는 항상 0부터 시작
- 모든 자료형에 대해서 배열로 구성 가능
- 구성 형태에 따라 1차원 배열, 2차원 배열, 3차원 배열, ...

1. 배열 : 1차원 배열

■ 1차원 배열 선언 형식

자료형	배열이름	[배열요소의 개수];
①	②	③

예: `int numbers[10];`

- ① 배열의 자료형을 선언한다. 배열 요소는 모두 자료형이 같아야 하고, 배열 요소의 자료형이 배열의 자료형이 된다.
- ② 변수 이름과 같은 규칙으로 정한다.
- ③ 대괄호([])를 사용해 배열 요소의 개수를 표시하는데, 배열 요소 개수가 배열 크기이다. 배열을 선언하면 메모리에 배열에 대한 공간이 할당되고 그 크기는 '자료형에 대한 메모리 할당 크기×배열 요소의 개수'이다.

그림 2-1 1차원 배열의 선언 형식

1. 배열 : 1차원 배열

– 예)

**각 자료형의 실제 바이트수는
시스템에 따라 다를 수 있음 !**

표 2-1 여러 자료형의 배열 선언 예와 의미

배열 선언 예	의미	배열 요소	메모리 할당 크기
char c[100];	char형 배열 요소 100개로 구성된 배열 c	c[0] ~ c[99]	1byte x 100
int i[100];	int형 배열 요소 100개로 구성된 배열 i	i[0] ~ i[99]	4byte x 100
short s[100];	short형 배열 요소 100개로 구성된 배열 s	s[0] ~ s[99]	2byte x 100
long l[100];	long형 배열 요소 100개로 구성된 배열 l	l[0] ~ l[99]	4byte x 100

1. 배열 : 1차원 배열

- [예제 2-1] 자료형에 대한 메모리 할당 크기 확인하기 : [교재 70p](#)

```
char c 크기 = 1   : char c_array 크기 = 100
int i 크기 = 4    : int i_array 크기 = 400
short s 크기 = 2  : short s_array 크기 = 200
float f 크기 = 4   : float f_array 크기 = 400
long l 크기 = 4    : long l_array 크기 = 400
```

- 배열 선언과 메모리 할당 구조 예

```
int mid_score[40];
```



그림 2-2 배열 선언과 메모리 할당 구조 예 : 40명의 중간고사 점수

1. 배열 : 1차원 배열

예제 2-1 자료형에 대한 메모리 할당 크기 확인하기

```

01  #include <stdio.h>
02
03  void main() {
04      char c, c_array[100];
05      int i, i_array[100];
06      short s, s_array[100];
07      float f, f_array[100];
08      long l, l_array[100];
09
10      printf(
11      printf("
12      printf("
13      printf("
14      printf("
15
16      getchar(); // 실행 창이 닫히지 않게 하기 위해 편의상 추가한 입력 대기 명령
17  }
```

TIP `getchar()` 컴파일하여 생성된 실행 파일(.exe)을 더블클릭해 실행하면 `main()` 함수가 실행되고, `main()` 함수가 끝나면 실행 창이 닫힌다. 그런데 실행 시간이 짧아 결과를 확인할 새도 없이 창이 닫힌다. 실행 결과를 확인할 수 있도록 `main()` 함수가 끝나기 직전에 입력 함수인 `getchar()`를 추가하여 입력 대기 상태로 만들 수 있다. `getchar()` 대신에 `system("pause")`를 사용할 수도 있지만 `#include <windows.h>`를 추가해야 하고 전처리기에 의해 헤더 파일이 복사되어 추가되므로 소스 파일이 커진다는 것을 알아두어야 한다.

1. 배열 : 1차원 배열

- 1차원 배열의 초기화
 - 1차원 배열의 초기화 형식

자료형 배열이름[배열크기] = { 초깃값 리스트 };

그림 2-3 1차원 배열의 초기화

- 1차원 배열의 초기화 예 1

`int A[5] = {1, 2, 3, 4, 5};`

또는

`int A[ ] = {1, 2, 3, 4, 5};`

또는

`int A[5];`
`A[0] = 1;`
`A[1] = 2;`
`A[2] = 3;`
`A[3] = 4;`
`A[4] = 5;`

배열의 모든 원소에 초깃값을 주면
배열 크기 생략 가능

(a) 1차원 배열의 초기화

	A[0]	A[1]	A[2]	A[3]	A[4]
A	1	2	3	4	5

(b) 메모리 할당 구조

그림 2-4 1차원 배열의 초기화 예 1

1. 배열 : 1차원 배열

- 1차원 배열의 초기화 예 2

```
int A[5] = {1, 2, 3};
```

또는

```
int A[5];
A[0] = 1;
A[1] = 2;
A[2] = 3;
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	2	3	0	0

(a) 초깃값의 개수 < 배열 크기

```
int A[3] = {1, 2, 3, 4, 5};
```

또는

```
int A[3];
A[0] = 1;
A[1] = 2;
A[2] = 3;
A[3] = 4;
A[4] = 5;
```



A[0]	A[1]	A[2]
1	2	3

(b) 초깃값의 개수 > 배열 크기

그림 2-5 1차원 배열의 초기화 예 2

1. 배열 : 1차원 배열

- [예제 2-2] 학년별 취득 학점 입출력하기 : [교재 72p](#)

*** 학년별 취득 학점 ***

1학년 : 총점 = 91, 등급 = A

2학년 : 총점 = 86, 등급 = B

3학년 : 총점 = 97, 등급 = A

1. 배열 : 1차원 배열

예제 2-2 학년별 취득 학점 입출력하기

```

01  #include <stdio.h>
02
03  void main(){
04      int i;
05      // 1차원 배열 초기화
06      [-----]
07      [-----]
08
09      printf("\n *** 학년별 취득 학점 *** \n\n");
10
11      [-----]
12      [-----]
13      [-----]
14
15      getchar();
16  }

```

1. 배열 : 1차원 배열 [실습]

- [예제 2-3] 입력한 숫자를 구구단으로 출력하기 : [교재 73p](#)

```

1~9의 정수를 입력하세요 : 7

7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
    
```

1. 배열 : 1차원 배열

예제 2-3 입력한 숫자를 구구단으로 출력하기

```

01  #include <stdio.h>
02
03  void main() {
04      int i = 0, n;
05      int multiply[9];   결과 값 저장 변수
06
07      printf("\n1~9의 정수를 입력하세요 : ");
08
09      [ ]
10
11
12
13
14
15
16
17
18
19
20
21
22
23      getchar(); getchar();
24  }

```

1. 배열 : 1차원 배열

- 문자 배열
 - 문자의 나열
 - “와 ”사이에 표시
 - 문자열을 저장하기 위해서는 문자열을 구성하는 문자들을 연속적으로 저장해야 하기 때문에 char형 배열을 사용
 - 배열의 자료형은 문자 자료형(char)
 - 문자 배열의 초기화는 문자열 그대로 지정하거나 초기값 문자 리스트 사용

1. 배열 : 1차원 배열

- 문자배열을 문자열 "String" 으로 초기화하는 예

```
char s1[10] = "String";
```

문자열 " " 뒤에는
NULL 문자가 추가된다.

	s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]	s1[6]	s1[7]	s1[8]	s1[9]
s1	S	t	r	i	n	g	\0			

(a) 문자열을 사용한 초기화

```
char s2[10] = { 'S', 't', 'r', 'i', 'n', 'g' };
```

	s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]	s2[6]	s2[7]	s2[8]	s2[9]
s2	S	t	r	i	n	g				

(b) 초기값 문자 리스트를 사용한 초기화

그림 2-7 문자 배열의 선언과 메모리 할당 구조

- s1을 초기값 문자 리스트를 사용 초기화

```
char s1[] = "String";
```



```
char s1[] = { 'S', 't', 'r', 'i', 'n', 'g', '\0' };
```

그림 2-8 문자열을 사용한 초기화 → 문자 리스트를 사용한 초기화

1. 배열 : 1차원 배열

- [예제 2-4] 문자 배열에 문자열을 저장하고 출력하기 : [교재 75p](#)

문자 배열 str[] : Data Structure!

예제 2-4 문자 배열에 문자열을 저장하고 출력하기

```

01  #include <stdio.h>
02
03  void main() {
04      char str[20] = "Data Structure!";
05      int i;
06      printf("\n문자 배열 str[] : ");
07      for (i = 0; str[i]; i++) {
08          printf("%c", str[i]);
09      }
10      getchar();
11  }
```


1. 배열 : 1차원 배열 [실습]

- [예제 2-5] 입력한 문자열의 길이 계산하기 : [교재 76p](#)

문자열을 입력하세요 : Korea!

입력된 문자열은
"Korea!"
입니다.

입력된 문자열의 길이 = 6

1. 배열 : 1차원 배열 [실습]

예제 2-5

입력한 문자열의 길이 계산하기

```

01  #include <stdio.h>
02
03  void main() {
04      int i, length = 0;
05      char str[50];
06      printf("\n문자열을 입력하세요 : ");
07      [
08          printf("\n입력된 문자열은 \n \n");
09      ]
10      [
11      ]
12      [
13          printf("\n \n입니다.");
14          printf("\n\n입력된 문자열의 길이 = %d \n", length);
15
16      getchar();
17  }

```

문자열을 입력하세요 : Korea!

입력된 문자열은
"Korea!"
입니다.

입력된 문자열의 길이 = 6

1. 배열 : 다차원 배열

- 다차원 배열
 - 2차원 이상의 배열
- 다차원 배열의 선언
 - 배열의 차수 만큼 [배열크기] 항목을 추가
 - 2차원 배열의 선언 형식

예: `int numbers[10][20];`

자료형 배열이름 [배열크기] [배열크기]

행 개수

열 개수

↓
행 번호

↓
열 번호

} 논리적 구조에서의 표현

그림 2-9 2차원 배열의 선언 형식

1. 배열 : 다차원 배열

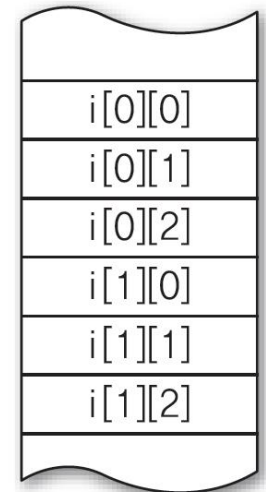
- 2차원 배열 선언의 논리적 구조와 물리적 구조 예

```
int i[2][3];
```

(a) 배열 선언

	열 번호 0	열 번호 1	열 번호 2
행 번호 0	i[0][0]	i[0][1]	i[0][2]
행 번호 1	i[1][0]	i[1][1]	i[1][2]

(b) 논리적 구조



(c) 물리적 구조

그림 2-10 2차원 배열의 선언과 논리적/물리적 구조 예

1. 배열 : 다차원 배열

- 3차원 배열의 선언 형식

자료형 배열이름 [배열크기] [배열크기] [배열크기]
 면 개수 행 개수 열 개수 → 논리적 구조에서의 표현

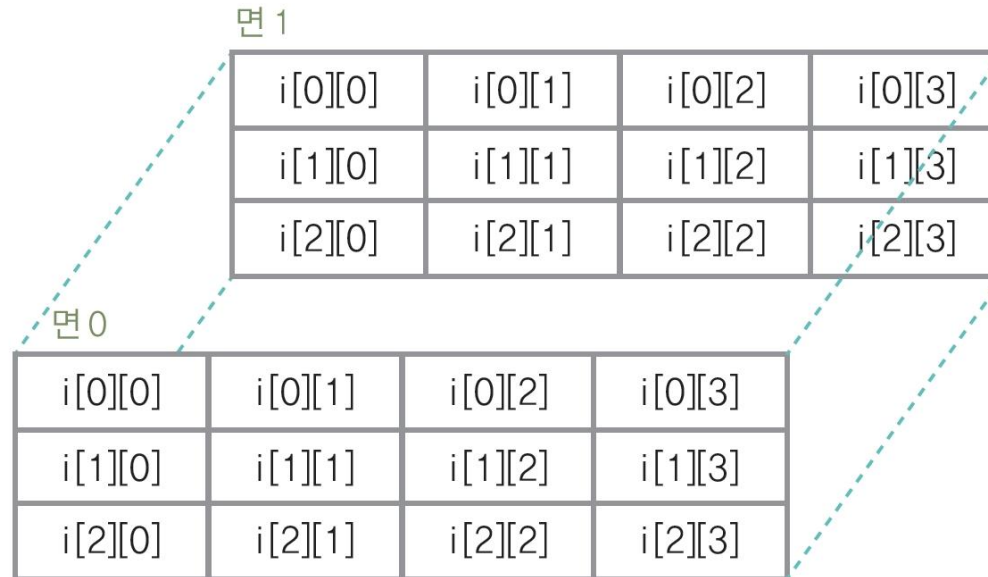
그림 2-11 3차원 배열의 선언 형식

1. 배열 : 다차원 배열

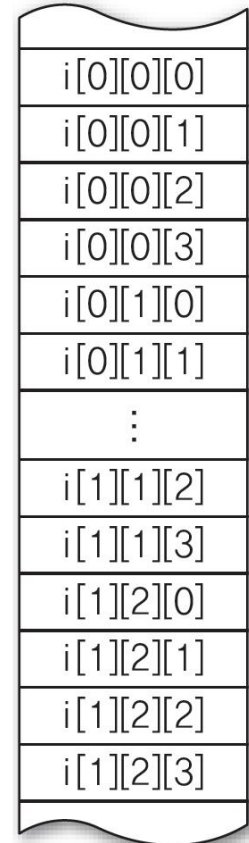
- 3차원 배열 선언 예

```
int i[2][3][4];
```

(a) 선언 형식



(b) 논리적 구조



(c) 물리적 구조

그림 2-12 3차원 배열의 선언과 논리적/물리적 구조 예

1. 배열 : 다차원 배열

■ 다차원 배열의 초기화

- 초기값의 지정형태는 다차원 배열이 배열의 배열이라는 것을 생각하여 초기값을 구분하여 지정하거나, 1차원 배열처럼 초기값 리스트를 지정하여 순서대로 배열요소의 초기값으로 설정
- 2차원 배열의 초기화와 논리적 구조

`int i[2][3] = {{1, 2, 3}, {4, 5, 6}};` 또는 `int i[2][3] = {1, 2, 3, 4, 5, 6};`

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6

그림 2-13 2차원 배열의 초기화와 논리적 구조

`int i[][3] = {{1, 2, 3}, {4, 5, 6}};` 또는 `int i[][3] = {1, 2, 3, 4, 5, 6};`

그림 2-14 행 크기를 생략한 2차원 배열의 초기화

1. 배열 : 다차원 배열

- 3차원 배열의 초기화와 논리적 구조

```
int i[2][3][4] = {{1, 2, 3, 4 }, {5, 6, 7, 8}, {9, 10, 11, 12},
                  {13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}};
```



그림 2-15 3차원 배열의 초기화와 논리적 구조

1. 배열 : 다차원 배열 [실습]

– [예제 2-6] 3차원 배열 입출력하기 : [교재 79p](#)

```
array [0] [0] [0] = 1
array [0] [0] [1] = 2
array [0] [0] [2] = 3
array [0] [0] [3] = 4
array [0] [1] [0] = 5
array [0] [1] [1] = 6
array [0] [1] [2] = 7
array [0] [1] [3] = 8
array [0] [2] [0] = 9
array [0] [2] [1] = 10
array [0] [2] [2] = 11
array [0] [2] [3] = 12
array [1] [0] [0] = 13
array [1] [0] [1] = 14
array [1] [0] [2] = 15
array [1] [0] [3] = 16
array [1] [1] [0] = 17
array [1] [1] [1] = 18
array [1] [1] [2] = 19
array [1] [1] [3] = 20
array [1] [2] [0] = 21
array [1] [2] [1] = 22
array [1] [2] [2] = 23
array [1] [2] [3] = 24
```

1. 배열 : 다차원 배열

예제 2-6 3차원 배열 입출력하기

```

01  #include <stdio.h>
02
03  void main() {
04      int array[2][3][4];
05      int i, j, k, value = 1;
06      ┌───────────────────────────────────────────────────────────────────────────────────┐
07      │                                                                                       │
08      │                                                                                       │
09      │                                                                                       │
10      │                                                                                       │
11      │                                                                                       │
12      │                                                                                       │
13      │                                                                                       │
14      │                                                                                       │
15      └───────────────────────────────────────────────────────────────────────────────────┘
16      getchar();
17  }
```

1. 배열 : 다차원 배열

■ 문자 다차원 배열

– 2차원 문자 배열의 선언 예

```
char c[3][20];
```

그림 2-16 2차원 문자 배열의 선언 예

– 2차원 문자 배열의 문자열 저장과 논리적 구조

(a)

```
char c[3][20]={ "Hong Gil Dong",  
                "Computer Department",  
                "Seoul Korea"};
```

(b)

```
strcpy(c[0], "Hong Gil Dong");  
strcpy(c[1], "Computer Department");  
strcpy(c[2], "Seoul Korea");
```

(c)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]
[0]	H	o	n	g		G	i	l		D	o	n	g	\0						
[1]	C	o	m	p	u	t	e	r		D	e	p	a	r	t	m	e	n	t	\0
[2]	S	e	o	u	l		K	o	r	e	a	\0								

그림 2-17 2차원 문자 배열의 문자열 저장과 논리적 구조

1. 배열 : 다차원 배열

- [예제 2-7] 3차원 배열을 이용해 문자 배열 입출력하기 : [교재 80p](#)

```

학생 1의 이름 : Hong Gil Dong
학생 1의 학과 : Computer Electronic
학생 1의 학번 : 202100101

학생 2의 이름 : Hong Gil Soon
학생 2의 학과 : Computer Science
학생 2의 학번 : 202100201

학생 1
    Hong Gil Dong
    Computer Electronic
    202100101

학생 2
    Hong Gil Soon
    Computer Science
    202100201
    
```

1. 배열 : 다차원 배열

예제 2-7

3차원 배열을 이용해 문자 배열 입출력하기

```

01 #include <stdio.h>
02
03 void main() {
04     int i, j, k;
05     char student[2][3][20];
06     for (i = 0; i < 2; i++) {
07         printf("\n 학생 %d의 이름 : ", i + 1);
08         gets(student[i][0]);
09         printf(" 학생 %d의 학과 : ", i + 1);
10         gets(student[i][1]);
11         printf(" 학생 %d의 학번 : ", i + 1);
12         gets(student[i][2]);
13     }
14
15     for (i = 0; i < 2; i++) {
16         printf("\n\n 학생 %d", i + 1);
17         for (j = 0; j < 3; j++) {
18             printf("\n\t");
19             for (k = 0; student[i][j][k] != '\0'; k++) {
20                 printf("%c", student[i][j][k]);
21             }
22         }
23     }
24     getchar();
25 }

```

학생 1의 이름 : Hong Gil Dong
 학생 1의 학과 : Computer Electronic
 학생 1의 학번 : 202100101

학생 2의 이름 : Hong Gil Soon
 학생 2의 학과 : Computer Science
 학생 2의 학번 : 202100201

학생 1
 Hong Gil Dong
 Computer Electronic
 202100101

학생 2
 Hong Gil Soon
 Computer Science
 202100201

2. 포인터 : 개념

■ 포인터 개념

- 변수의 메모리 주소값
- 포인터변수
 - 주소값을 저장하는 특별한 변수
 - 포인터 변수가 어떤 변수의 주소를 저장하고 있다는 것은 포인터 변수가 그 변수를 가리키고 있다(포인팅하고 있다)는 의미
 - 포인터 변수를 이용하여, 연결된 주소의 변수 영역을 액세스 함
 - 포인터변수를 간단히 포인터라고 함

2. 포인터 : 개념

- 포인터의 액세스 처리

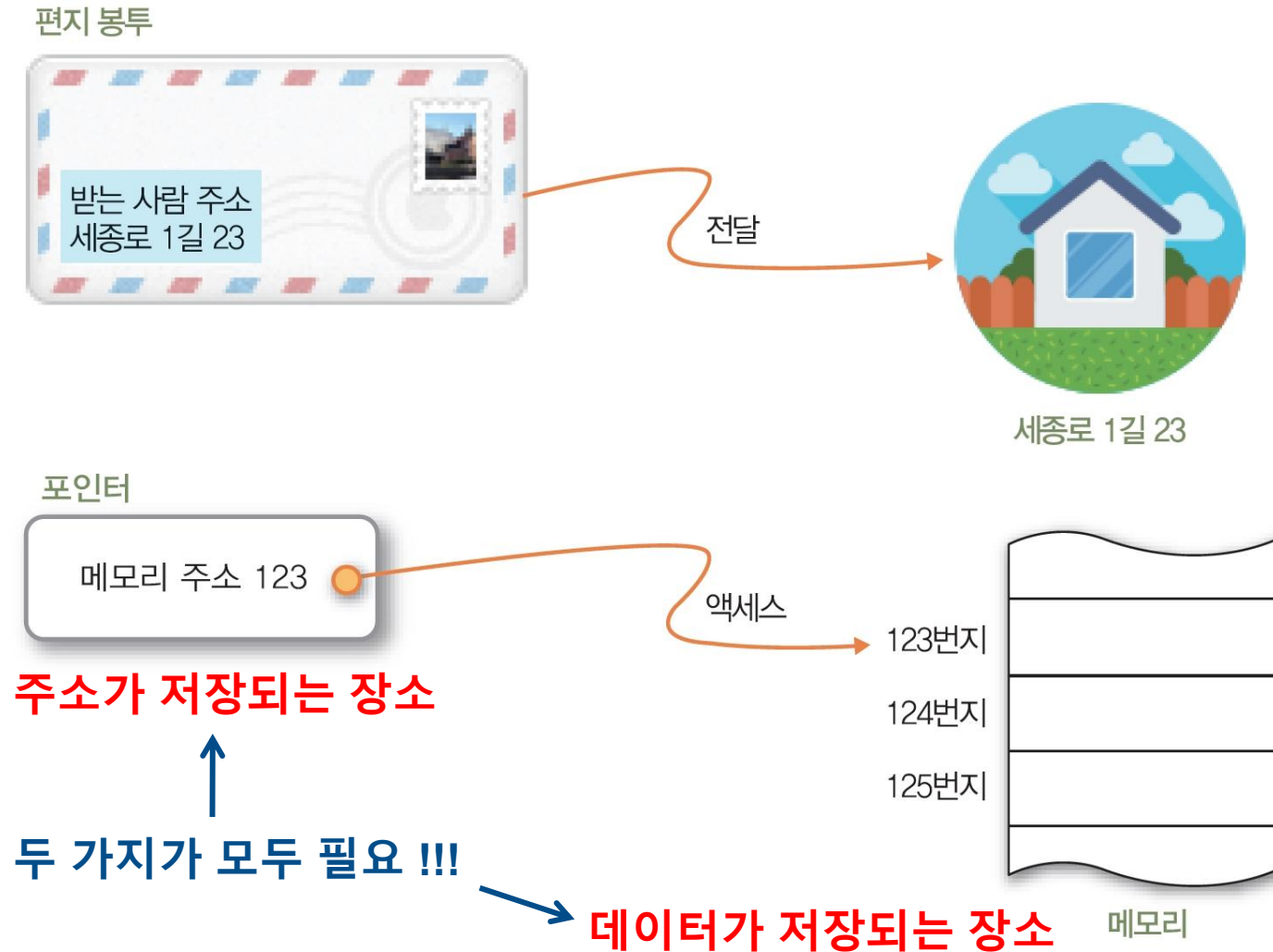


그림 2-18 포인터의 액세스 처리

2. 포인터

- 포인터 사용 예

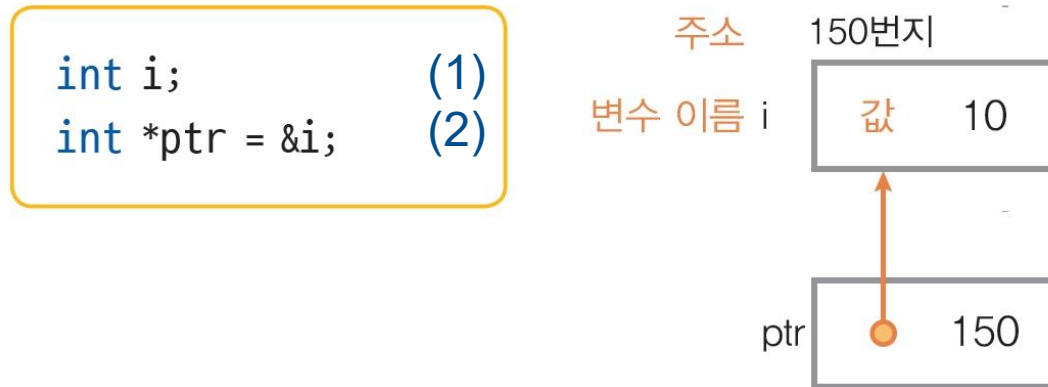


그림 2-19 포인터의 사용 예

- ①에서 int형 변수 i를 선언했을 때에 i 변수의 메모리 번지를 150번지라고 한다면,
- ②에서 변수 i의 주소값 (150) 을 포인터변수 ptr에 저장.

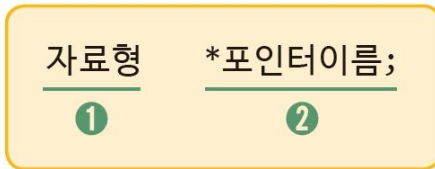
ptr에는 메모리 주소 150이 저장되므로,

포인터변수 ptr은 150번지의 변수 i를 가리키는(지시하는) 상태가 됨

2. 포인터 : 포인터 선언

■ 포인터 선언

– 포인터 선언 형식



- ① 포인터 자체의 자료형이 아니라 포인터에 저장할 주소에 있는 일반 변수의 자료형이다.
- ② 일반 변수 이름과 구별되도록 앞에 *를 붙여 포인터임을 나타낸다.

그림 2-20 포인터의 선언 형식

예) `int *ptr;`

2. 포인터 : 포인터 선언

- 포인터를 다양한 자료형으로 선언한 예

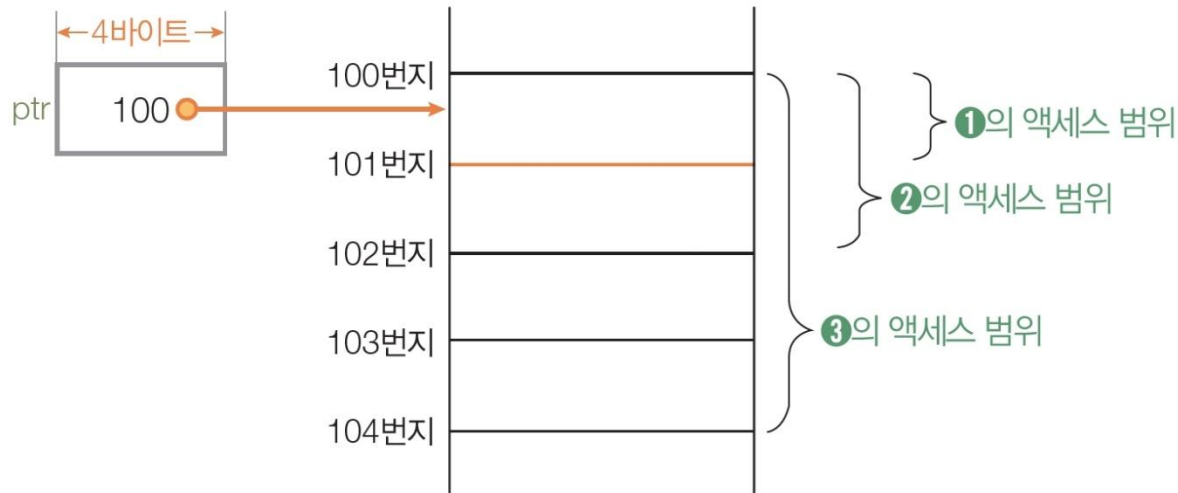
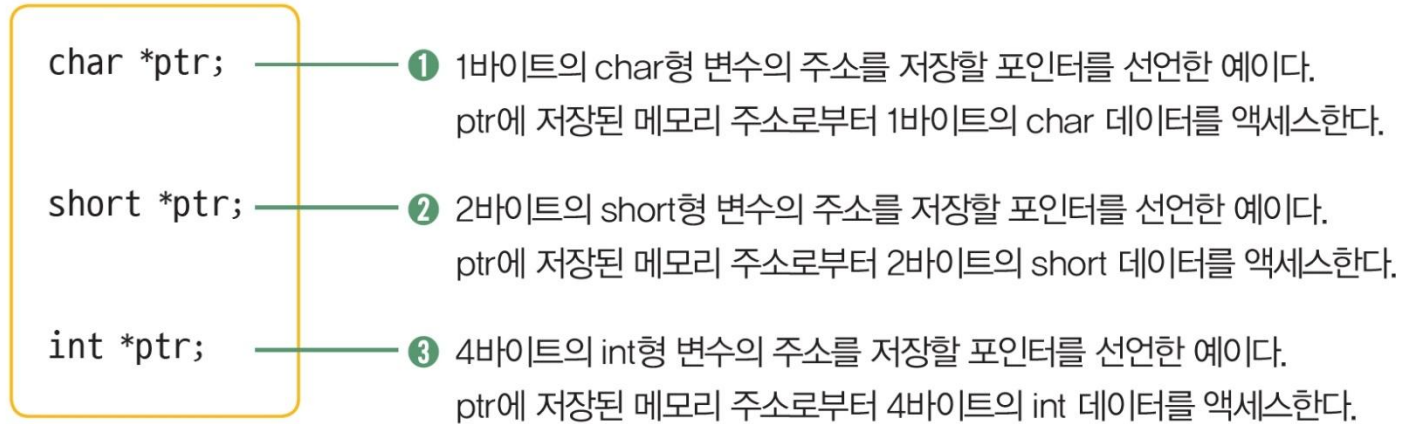


그림 2-21 포인터에서 선언한 자료형에 따른 메모리 액세스 범위

2. 포인터 : 포인터 연산

■ 포인터 연산자

– 주소 연산자 : &

- 변수의 주소 얻기 위해 사용
- 형식

포인터 = &변수;

그림 2-22 주소 연산자 사용 형식

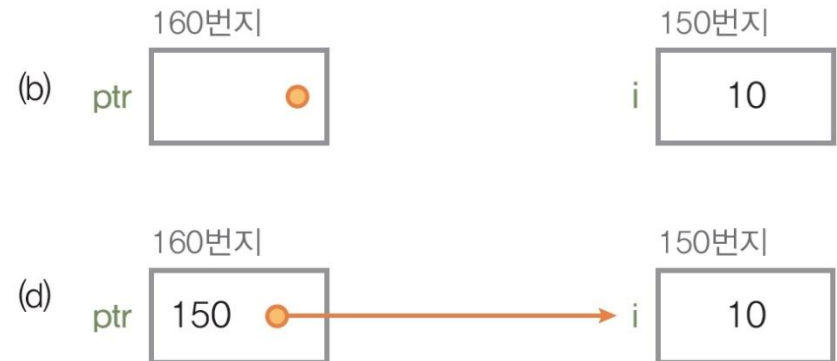
• 예

(a) `int i = 10;`
`int *ptr;`

(c) `ptr = &i;`

그림 2-23 포인터 선언과 사용 예

*ptr 의 값 : (데이터)
ptr 의 값 : (주소)



2. 포인터 : 포인터 연산

– 참조 연산자 : *

- 저장된 주소에 있는 값(변수에 저장된 값)을 액세스하는 연산자
- 형식

① *포인터 = 값;

② 변수 = *포인터;

데이터 값을 포인터가 가리키고 있는 주소에 저장

포인터가 가리키는 주소에 있는 값을 변수에 저장

그림 2-24 참조 연산자 사용 형식

2. 포인터 : 포인터 연산

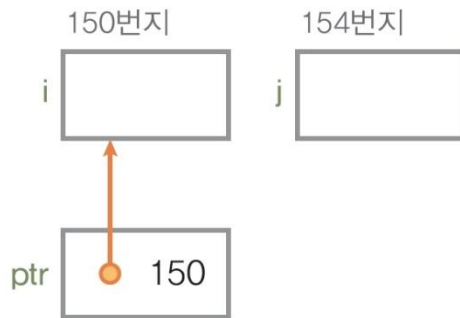
• 예

```
int i, j;
int *ptr;
ptr = &i;

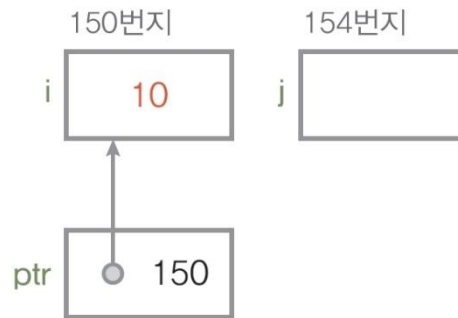
*ptr = 10;

j = *ptr;
```

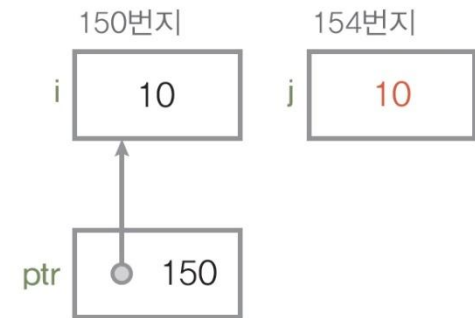
- 주소 연산자를 사용하여 변수 i의 주소를 포인터 ptr에 할당한다. 포인터 ptr은 변수 i를 가리킨다.
- 참조 연산자를 사용하여 포인터 ptr이 가리키는 영역에 값 10을 지정한다. 따라서 변수 i에 10이 저장된다.
- 다시 참조 연산자를 사용하여 ptr이 가리키는 영역의 값을 변수 j에 지정한다. 따라서 ptr이 가리키는 변수 i의 값인 10을 변수 j에 저장한다.



①의 실행 결과



②의 실행 결과



③의 실행 결과

그림 2-25 포인터 연산자를 사용한 예

2. 포인터 : 포인터 연산 [실습]

- [예제 2-8] 포인터 연산자를 이용해 변수 액세스하기 : [교재 84p](#)

```

i의 값 = 10
j의 값 = 20
i의 메모리 주소(&i) = 1245052
j의 메모리 주소(&j) = 1245048

<< ptr=&i 실행 >>
ptr의 메모리 주소(&ptr) = 1245044
ptr의 값(ptr) = 1245052
ptr의 참조값(*ptr) = 10

<< ptr=&j 실행 >>
ptr의 메모리 주소(&ptr) = 1245044
ptr의 값(ptr) = 1245048
ptr의 참조값(*ptr) = 20

<< i=*ptr 실행 >>
i의 값 = 20
    
```

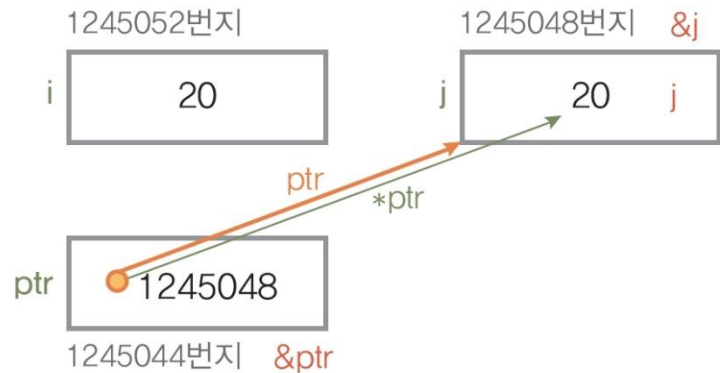


그림 2-26 [예제 2-8]을 실행한 결과

2. 포인터 : 포인터 연산

예제 2-8 포인터 연산자를 이용해 변수 액세스하기

```

01  #include <stdio.h>
02
03  void main() {
04      int i = 10, j = 20;
05      int *ptr;
06      ┌───────────────────────────────────┐
07      │                                   │
08      │                                   │
09      │                                   │
10      │                                   │
11      │                                   │
12      │                                   │
13      │                                   │
14      │                                   │
15      │                                   │
16      │                                   │
17      │                                   │
18      │                                   │
19      │                                   │
20      │                                   │
21      │                                   │
22      │                                   │
23      │                                   │
24      └───────────────────────────────────┘
25
26      getchar();
27  }

```

i의 값 = 10
 j의 값 = 20
 i의 메모리 주소(&i) = 1245052
 j의 메모리 주소(&j) = 1245048

<< ptr=&i 실행 >>
 ptr의 메모리 주소(&ptr) = 1245044
 ptr의 값(ptr) = 1245052
 ptr의 참조값(*ptr) = 10

<< ptr=&j 실행 >>
 ptr의 메모리 주소(&ptr) = 1245044
 ptr의 값(ptr) = 1245048
 ptr의 참조값(*ptr) = 20

<< i=*ptr 실행 >>
 i의 값 = 20

2. 포인터 : 포인터 연산

■ 포인터의 초기화

– 일반 변수를 초기화하는 방법과 같음

- 포인터에 주소 지정 방법

① 주소 연산자를 사용하여 변수 주소를 지정

```
int i;  
int *ptr = &i;
```

② 동적 메모리를 할당하고 그 시작 주소를 포인터 값으로 지정

```
char *ptr = (char *)malloc(100);
```


2. 포인터 : 포인터 연산

- ③ 문자형 포인터에 문자열의 시작 주소를 지정

```
char *ptr = "korea";
```

- ④ 배열 이름을 이용하여 배열 시작 주소를 지정

```
char A[100];  
char *ptr = A;
```

- ⑤ 배열의 첫 번째 요소의 주소를 이용하여 배열 시작 주소를 지정

```
char A[100];  
char *ptr = &A[0];
```

2. 포인터 : 포인터 연산 [실습]

■ 포인터와 문자 배열

– [예제 2-9] : 포인터를 이용해 문자열 처리하기 : [교재 87p](#)

```
string1의 주소 = 1374236    ptr1 = 1374236
string1 = Dreams come true!
ptr1 = Dreams come true!

come true!
come true!

!eurt emoc smaerD

string1 = Dreams come true!
string2 = Dreams come true!

string1 = Peaces come true!
```

2. 포인터 : 포인터 연산

예제 2-9

포인터를 이용해 문자열 처리하기

```

01  #include <stdio.h>
02
03  void main() {
04      int i;
05      char string1[20] = "Dreams come true!", string2[20], *ptr1, *ptr2;
06
07      -----
08
09
10
11
12
13
14
15
16
17
18
19

```

string1의 주소 = 1374236 ptr1 = 1374236
 string1 = Dreams come true!
 ptr1 = Dreams come true!

come true!
 come true!

!eurt emoc smaerD

string1 = Dreams come true!
 string2 = Dreams come true!

string1 = Peaces come true!

2. 포인터 : 포인터 연산

```

20  printf("\n\n string1 = %s", string1);
21  |-----|
22  |
23  |
24  |
25  |
26  |
27  |
28  |
29  |-----|
30  getchar();
31  }

```

```

string1의 주소 = 1374236    ptr1 = 1374236
string1 = Dreams come true!
ptr1 = Dreams come true!

come true!
come true!

leurt emoc smaerD

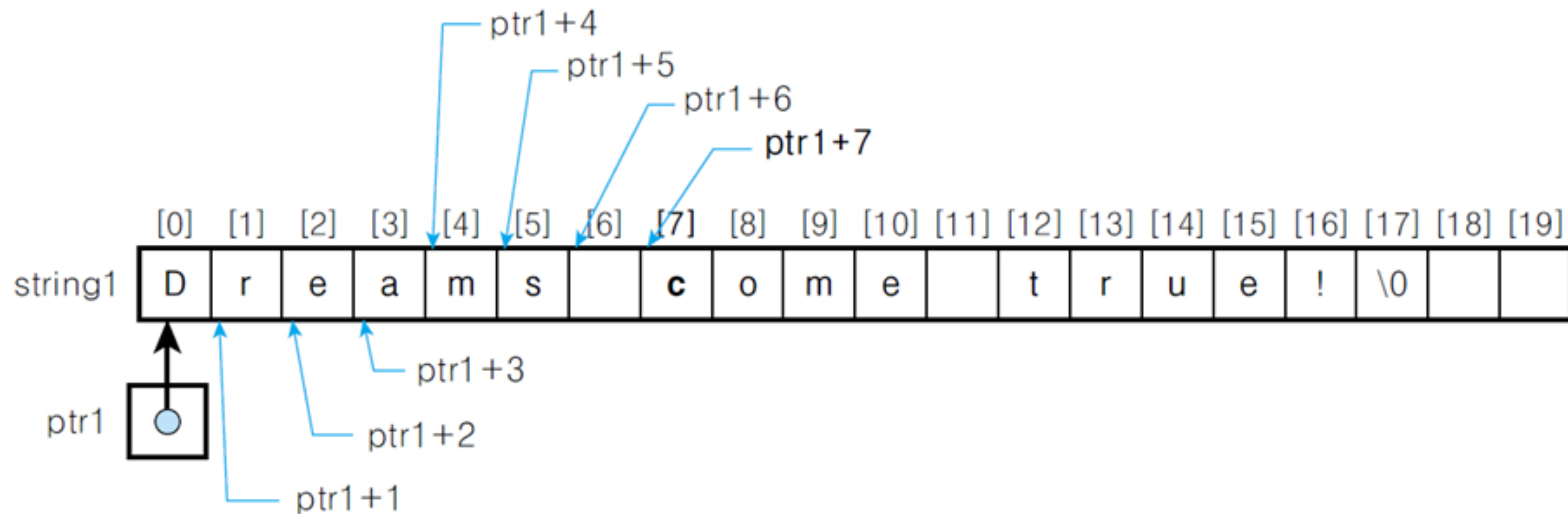
string1 = Dreams come true!
string2 = Dreams come true!

string1 = Peaces come true!

```

2. 포인터 : 포인터 연산

- [예제 2-9] : 포인터를 이용한 문자열 처리 프로그램
 - $\text{ptr1}+7$ 과 $\&\text{string1}[7]$ 의 관계

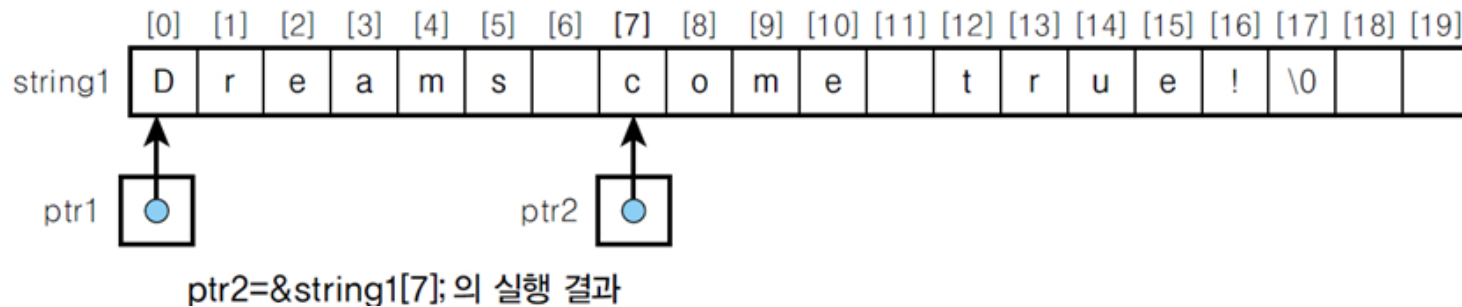


$\text{ptr1}+7$ 과 $\&\text{string1}[7]$ 의 관계

- *(ptr1+7) 은 `string1[7]` 의 값이므로 문자 'c' 가 된다.
- `printf("%s", ptr1+7)` 를 수행하면 'c' 부터 문자열 끝까지 출력한다.

2. 포인터 : 포인터 연산

- [예제 2-9] : 포인터를 이용한 문자열 처리 프로그램
 - 문자 'c' 를 지시하는 포인터 지정
 - `ptr2 = &string1[7];`

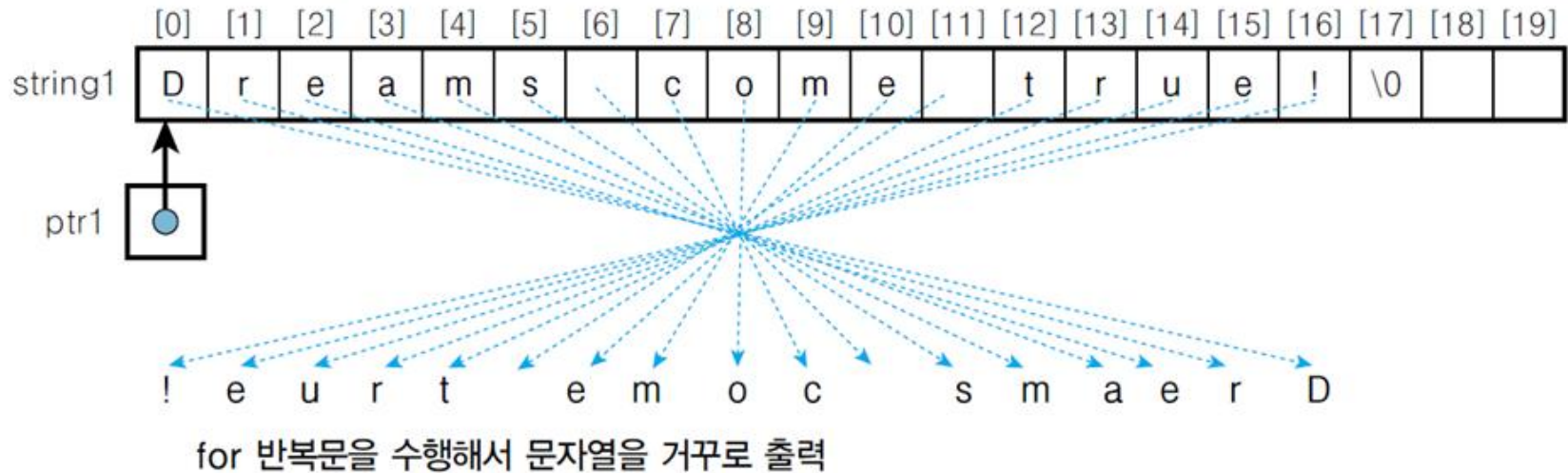


- `printf("%s", ptr2)` 를 수행하면 'c' 부터 문자열 끝까지 출력한다.

2. 포인터 : 포인터 연산

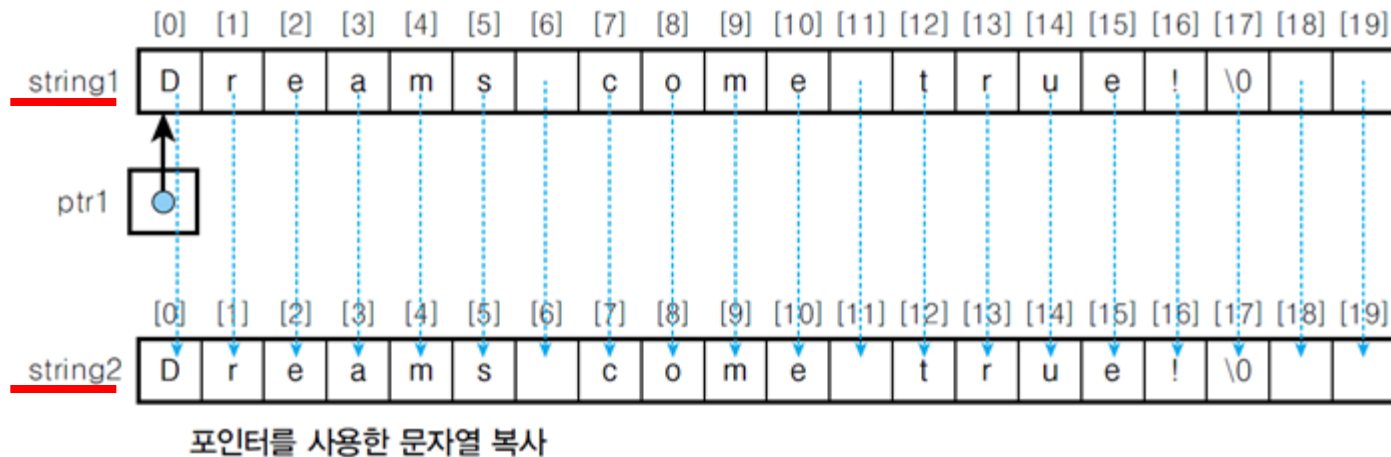
- [예제 2-9] : 포인터를 이용한 문자열 처리 프로그램
- for 반복문을 사용하여 문자열을 거꾸로 출력

```
for(i=16; i>=0; i--){
    putchar(*(ptr1+i));
}
```



2. 포인터 : 포인터 연산

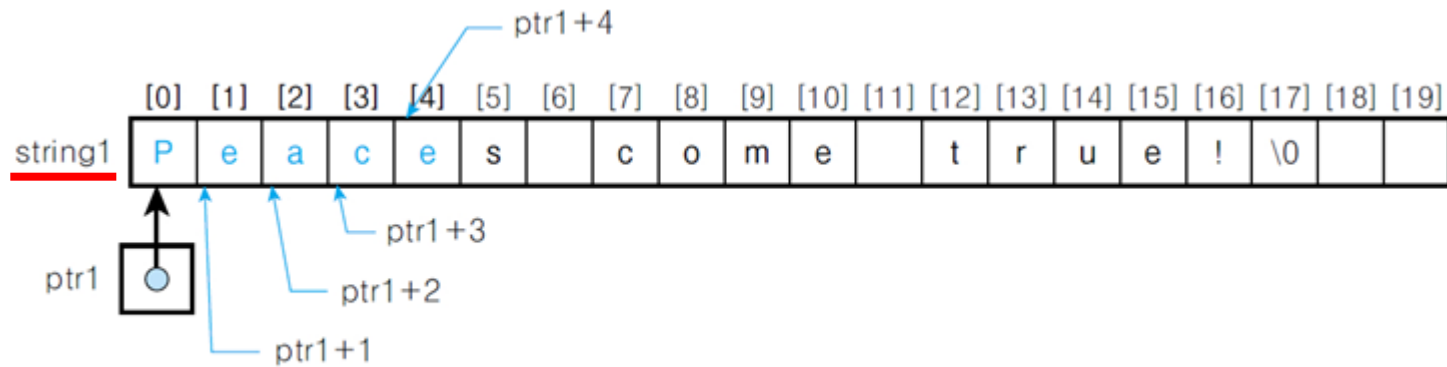
- [예제 2-9] : 포인터를 이용한 문자열 처리 프로그램
 - 포인터를 사용한 문자열 복사
 - for (i=0; i<20; i++)
 - string2[i] = *(ptr1+i)



2. 포인터 : 포인터 연산

- [예제 2-9] : 포인터를 이용한 문자열 처리 프로그램

- 포인터를 사용하여 string1의 문자열 변경
- `*ptr1 = 'P';`
- `*(ptr1+1) = 'e';`
- `*(ptr1+2) = 'a';`
- `*(ptr1+3) = 'c';`
- `*(ptr1+4) = 'e';`



포인터를 사용하여 string1의 문자열 변경

2. 포인터 : 포인터 연산

■ 포인터 배열

– 포인터 자료형을 배열로 구성

- 여러 개의 포인터를 하나의 배열로 구성한 배열의 특징과 포인터의 특징을 모두 활용할 수 있다.

– 포인터 배열의 선언형식

자료형 *포인터배열이름 [배열크기];

그림 2-27 포인터 배열 선언 형식

- 포인터 배열에서 각 배열요소는 포인터
- 2차원 문자배열을 1차원 포인터배열로 표현
 - 2차원 배열의 행의 개수 : 포인터배열의 크기
 - 포인터배열의 각 배열요소 : 각 문자열에 대한 시작주소를 가진 포인터

2. 포인터 : 포인터 연산

- 2차원 배열과 1차원 포인터 배열의 비교

공간낭비 !

```
char string[3][10] = { "Dreams", "come", "true!" };
```

(a)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
string[0]	D	r	e	a	m	s	\0			
string[1]	c	o	m	e	\0					
string[2]	t	r	u	e	!	\0				

```
char *ptr[3] = { { "Dreams" }, { "come" }, { "true!" } };
```

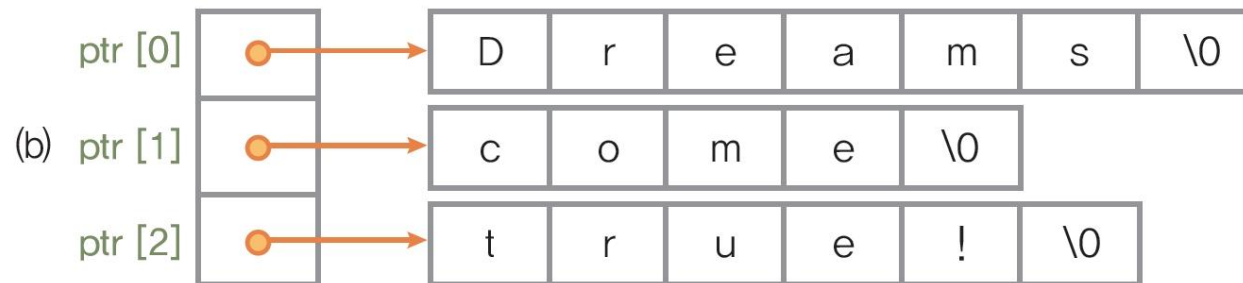
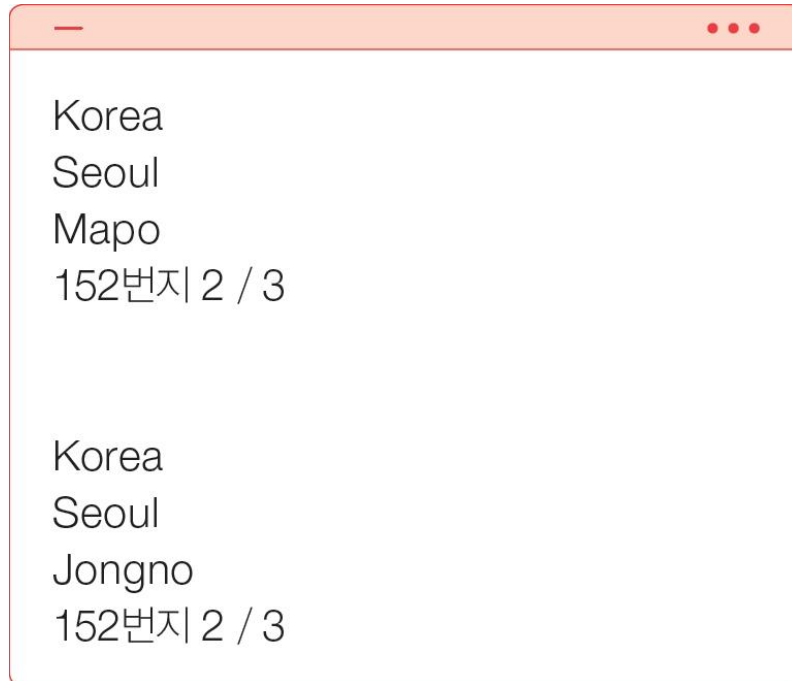


그림 2-28 2차원 배열과 1차원 포인터 배열의 비교

2. 포인터 : 포인터 연산

- [예제 2-10] : 포인터 배열을 이용해 문자열 저장하기 : [교재 90p](#)



2. 포인터 : 포인터 연산

예제 2-10 포인터 배열을 이용해 문자열 저장하기

```

01  #include <stdio.h>
02
03  void main() {
04      int i;
05      char *ptrArray[4] = { { "Korea" }, { "Seoul" }, { "Mapo" }, { "152번지 2 / 3" } };
06      |-----|
07      |
08      |
09      |
10      |
11      |
12      |
13      |-----|
14      getchar();
15  }

```

Korea
 Seoul
 Mapo
 152번지 2 / 3

Korea
 Seoul
 Jongno
 152번지 2 / 3

2. 포인터 : 포인터 연산

- 포인터의 포인터
 - 포인터를 가리키고 있는 포인터. 이중 포인터
 - 포인터의 포인터 선언 형식과 예

자료형 ******포인터이름;

(a) 선언 형식

그림 2-29 포인터의 포인터 선언 형식과 예

char **ptr



(b) 사용 예

2. 포인터 : 포인터 연산 [실습]

- [예제 2-11] : 포인터 배열과 포인터의 포인터 사용하기 : [교재 91p](#)

```
ptrArray[0]의 주소 (&ptrArray[0]) = 1245048
ptrArray[0]의 값 (ptrArray[0]) = 4338436
ptrArray[0]의 참조값 (*ptrArray[0]) = K
ptrArray[0]의 참조 문자열 (*ptrArray[0]) = Korea
```

```
ptrArray[1]의 주소 (&ptrArray[1]) = 1245052
ptrArray[1]의 값 (ptrArray[1]) = 4338428
ptrArray[1]의 참조값 (*ptrArray[1]) = S
ptrArray[1]의 참조 문자열 (*ptrArray[1]) = Seoul
```

```
ptrptr의 주소 (&ptrptr) = 1245044
ptrptr의 값 (ptrptr) = 1245048
ptrptr의 1차 참조값 (*ptrptr) = 4338436
ptrptr의 2차 참조값 (**ptrptr) = K
ptrptr의 2차 참조 문자열 (**ptrptr) = Kore
```

```
*ptrArray[0] : Korea
**ptrptr : Korea
```

```
*ptrArray[1] : Seoul
**(ptrptr+1) : Seoul
```

- 정수 n 개를 입력 받아 포인터를 이용하여 배열을 왼쪽으로 두 칸 회전시켜 출력하는 프로그램을 작성하시오.

- 예를 들어 배열이

- 1 2 3 4 5 6 7

라면 왼쪽으로 두 칸 회전한 결과는

- 3 4 5 6 7 2 3

이다.

- 반드시 포인터를 이용하여 배열을 처리할 것
- 배열 인덱스 `arr[i]` 사용 금지
- 출력 형식은 예시와 동일하게 할 것


```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main() {

    int n;
    int arr[100];

    printf("n 값을 입력하시오: ");
    scanf("%d", &n);

    printf("%d 개의 정수를 입력하시오.\n", n);

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int* p = arr;

    /* 포인터를 이용하여 배열을 왼쪽으로 두 칸 회전시키는 코드 작성 */

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

3. 구조체 : 개념

■ 구조체 개념

- 구조체도 배열처럼 여러 개의 데이터를 그룹으로 묶어서 하나의 자료형으로 정의하고 사용하는 자료형
 - 배열은 같은 자료형만을 그룹으로 묶을 수 있지만, 구조체는 서로 다른 자료형을 그룹으로 묶을 수 있으므로 복잡한 자료 형태를 정의하는데 유용하게 사용됨
- 여러 자료형의 필드를 가지고 있는 레코드를 만들 때 구조체 사용함
- 필드, 레코드, 파일의 개념

파일

김선달	2014년 입사	3500
이몽룡	2015년 입사	3000
홍길동	2015년 입사	3200
향단이	2016년 입사	2900

레코드

필드

그림 2-30 필드, 레코드, 파일의 개념

3. 구조체 : 구조체 선언

■ 구조체 선언

- 여러 자료형의 변수들을 그룹으로 묶어서 하나의 자료형으로 선언
- 구조체이름, 자료형, 데이터 항목으로 구성
 - 구조체의 이름 - 구조체로 정의하는 새로운 자료형의 이름
 - 항목 - 구조체를 구성하는 내부 변수들의 이름
 - 구조체의 항목은 배열의 각 배열요소에 해당
 - 배열요소는 모두 같은 자료형으로 되어있으므로 배열요소에 대한 선언 없이 사용이 가능하지만, 구조체에서는 각 항목이 다른 자료형을 가질 수 있기 때문에 항목별로 자료형과 항목이름(변수이름)을 선언해야 한다.

3. 구조체 : 구조체 선언

– 구조체형의 선언과 사용 형식

```
struct 구조체형이름 {
    자료형 항목1;
    자료형 항목2;
    ...
    자료형 항목n;
};
```

(a) 선언 형식

```
struct 구조체이름 구조체변수이름;
```

(b) 사용 형식

그림 2-31 구조체형의 선언과 사용 형식

- 구조체 선언은 사용할 구조체의 모양을 정의한 것뿐이므로
 - 구조체 변수를 사용하려면 구조체 변수를 다시 선언해야 한다.
- 구조체 사용 단계
 - ① 구조체형 선언 : 내부 구조를 정의한다.
 - ② 구조체 변수 선언 : 구조체형에 따른 변수를 선언한다.
 - ③ 구조체 변수의 사용 : 변수 내부 항목에 데이터를 저장하고 사용한다.

- 구조체 사용 예

```
struct employee {  
    char name[10];  
    int year;  
    int pay;  
};
```



그림 2-32 구조체형 employee 선언 예

```
struct employee Lee, Kim, Park;
```

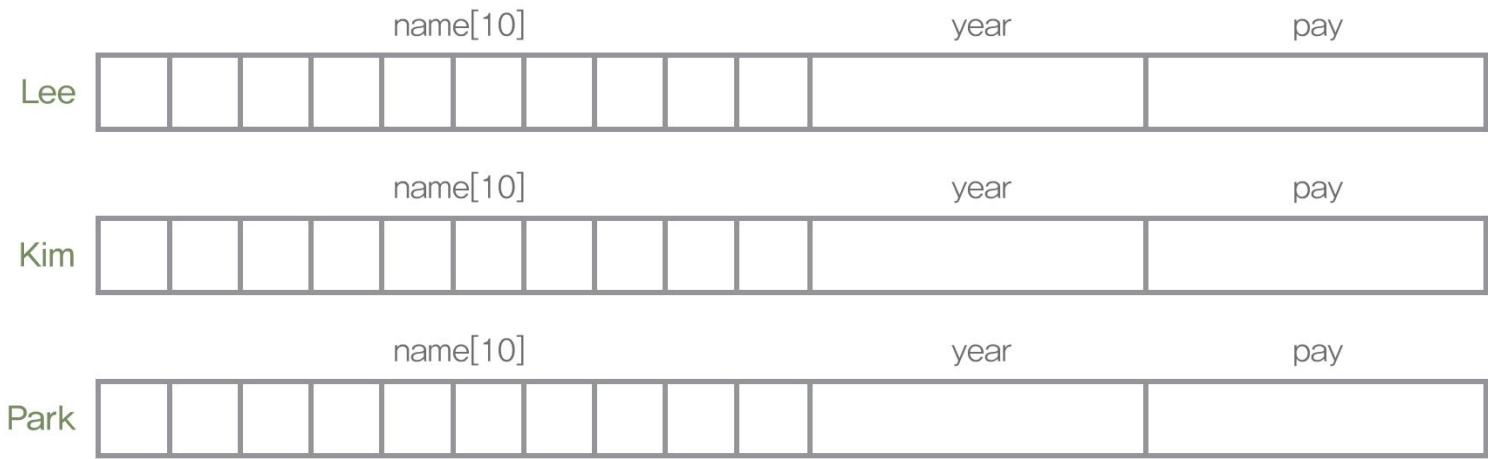


그림 2-33 구조체형 employee에 대한 구조체 변수 선언 예

3. 구조체 : 구조체 선언

- 구조체 변수의 선언 방법

표 2-2 구조체 변수의 선언 방법

방법	예
구조체형을 선언한 후에 구조체 변수 선언	<pre> struct employee { char name[10]; int year; int pay; }; struct employee Lee; </pre>
구조체형과 구조체 변수를 연결하여 선언	<pre> struct employee { char name[10]; int year; int pay; } Lee; </pre>
구조체형 이름을 생략하고 구조체 변수 이름만 선언	<pre> struct { char name[10]; int year; int pay; } Lee; </pre>

3. 구조체 : 구조체 변수의 초기화

■ 구조체의 변수의 초기화

- 일반 변수 초기화와 마찬가지로 구조체 변수 초기화하려면 구조체 변수를 선언하면서 변수의 초기값 지정
- 일반 변수는 값을 하나만 가지므로 초기값도 하나지만, 구조체는 내부 항목이 여러 개일 수 있으므로 내부 항목의 자료형과 개수를 순서에 맞추어 초기값 리스트로 지정하고 중괄호({ }) 사용
- 예

```
struct employee { // 구조체형 선언
    char name[10];
    int year;
    int pay;
};
```

```
struct employee Lee = { "Ann", 2022, 5200 }; // 구조체 변수의 초기화
```

	name[10]										year	pay
Lee	A	n	n	\0							2022	5200

그림 2-34 구조체 변수를 초기화한 예

3. 구조체 : 데이터 항목의 참조

■ 데이터 항목의 참조

- 구조체 연산자 사용 구조체 변수에 있는 각 데이터 항목 참조

표 2-3 구조체 연산자

종류	설명
점 연산자(.)	구조체 변수의 데이터 항목을 지정한다.
화살표 연산자(->)	구조체형 포인터에서 포인터가 가리키는 구조체 변수의 데이터 항목을 지정한다.

3. 구조체 : 데이터 항목의 참조

– 점 연산자 : .

- 구조체 변수에 있는 데이터 항목을 개별적으로 지정할 때 사용
- 예

① struct employee Lee;

② Lee.name = "Ann"; ← C 언어에서는 strcpy(Lee.name, "Ann") 로 사용해야 함 !

③ Lee.year = 2022;

④ Lee.pay = 5200;

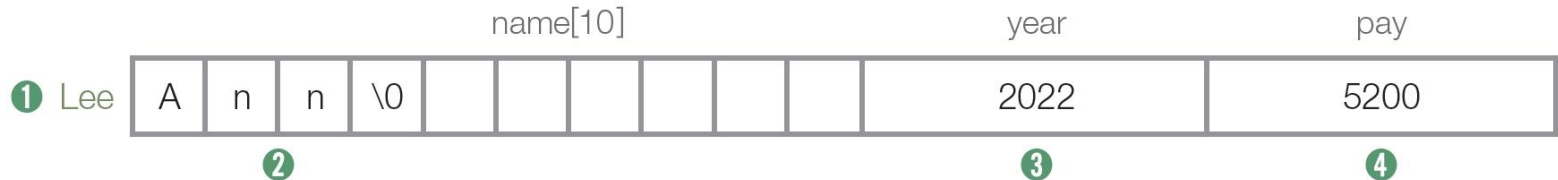


그림 2-35 점 연산자를 이용한 데이터 항목값 지정

3. 구조체 : 데이터 항목의 참조

- [예제 2-12] : 점 연산자를 이용해 데이터 항목 참조하기 : [교재 100p](#)

이름 : 이진호

입사 : 2022

연봉 : 5200

이름 : 이한영

입사 : 2023

연봉 : 4300

이름 : 이상원

입사 : 2023

연봉 : 4500

이름 : 이상범

입사 : 2024

연봉 : 3900

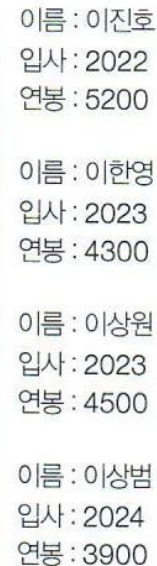
3. 구조체 : 데이터 항목의 참조

예제 2-12 점 연산자를 이용해 데이터 항목 참조하기

```

01  #include <stdio.h>
02  #include <string.h>
03
04  struct employee {
05      char name[10];
06      int year;
07      int pay;
08  };
09
10  void main() {
11      int i;
12      struct employee Lee[4] = {
13          { "이진호", 2022, 5200 },
14          { "이한영", 2023, 4300 },
15          { "이상원", 2023, 4500 },
16          { "이상범", 2024, 3900 }
17      };
18
19      [ ]
20      [ ]
21      [ ]
22      [ ]
23      [ ]
24      getchar();
25  }

```



이름 : 이진호
입사 : 2022
연봉 : 5200

이름 : 이한영
입사 : 2023
연봉 : 4300

이름 : 이상원
입사 : 2023
연봉 : 4500

이름 : 이상범
입사 : 2024
연봉 : 3900

3. 구조체 : 데이터 항목의 참조

- 화살표 연산자 : ->

- 구조체 포인터 변수에서 포인터가 가리키는 구조체 변수의 데이터 항목을 지정하기 위해서 화살표 연산자 사용

• 예

① `struct employee Kim;`

② `struct employee *Sptr = &Kim;`

③ `Sptr->name = "susan";` ← `strcpy(Sptr->name, "susan")` 로 사용해야 함 !

④ `Sptr->year = 2022;`

⑤ `Sptr->pay = 5300;`

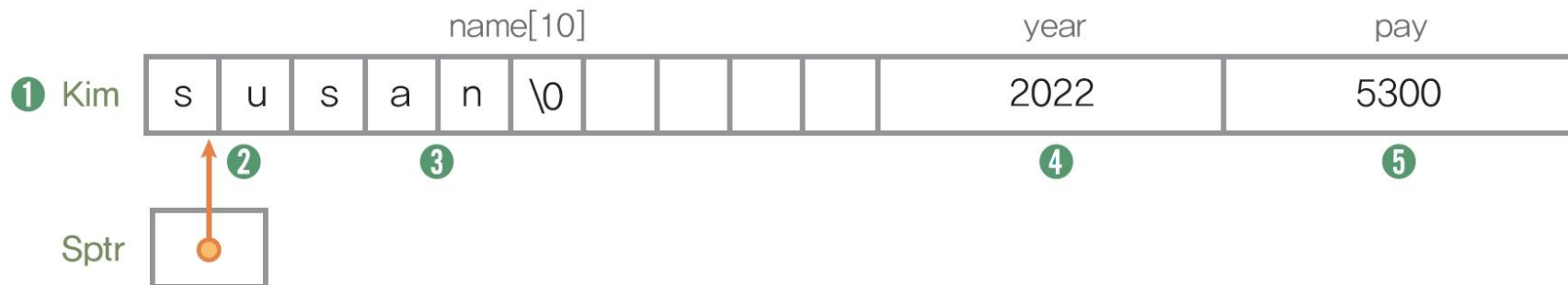


그림 2-36 화살표 연산자를 이용한 데이터 항목값 지정

3. 구조체 : 데이터 항목의 참조

- 구조체 포인터를 이용한 데이터 항목 지정 방법

```
Sptr->name = "susan";
Sptr->year = 2022;
Sptr->pay = 5300;
```

(a) 구조체 포인터의 화살표 연산자 사용



```
(*Sptr).name = "susan";
(*Sptr).year = 2022;
(*Sptr).pay = 5300;
```

(b) 구조체 포인터의 참조 연산자 사용

그림 2-37 구조체 포인터를 이용한 데이터 항목 지정 방법

괄호를 넣지 않으면 오류 발생

strcpy(Sptr->name, "susan") 로 사용해야 함 !

3. 구조체 : 데이터 항목의 참조

- [예제 2-13] : 화살표 연산자를 이용해 데이터 항목 참조하기 : [교재 101p](#)

이름 : 이순신
입사 : 2023
연봉 : 6900

3. 구조체 : 데이터 항목의 참조

예제 2-13 화살표 연산자를 이용해 데이터 항목 참조하기

```

01 #define _CRT_SECURE_NO_WARNINGS
02 #include <stdio.h>
03 #include <string.h>
04
05 struct employee {
06     char name[10];
07     int year;
08     int pay;
09 };
10
11 void main() {
12     struct employee Lee;
13     struct employee *Sptr = &Lee;
14     _____
15     _____
16     _____
17
18     printf("\n 이름 : %s", Sptr->name);
19     printf("\n 입사 : %d", Sptr->year);
20     printf("\n 연봉 : %d", Sptr->pay);
21
22     getchar();
23 }

```



```

이름 : 이순신
입사 : 2023
연봉 : 6900

```

3. 구조체 : 구조체 연산

■ 데이터 항목 참조 연산

- 점 연산자와 화살표 연산자 이용해 구조체 데이터 항목 개별적으로 참조
- 예

```
struct employee Lee;
struct employee *Sptr;
Sptr = &Lee;
Lee.year = 2023;
Sptr->pay = 4000;
Sptr->name = "Ann";
```

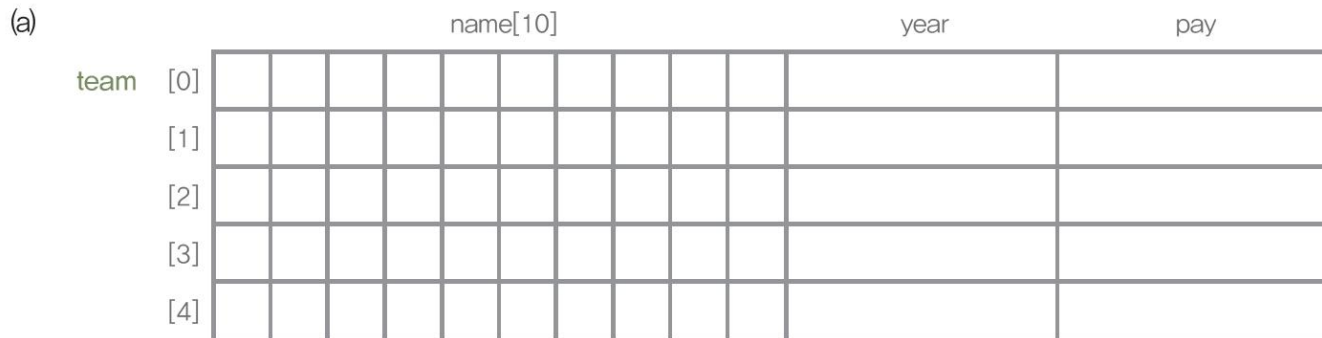
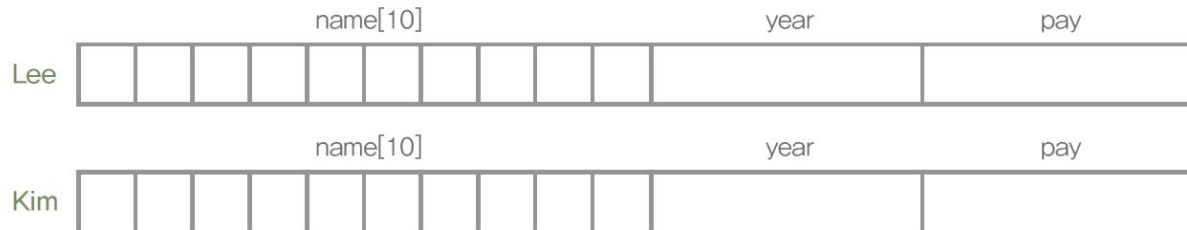
strcpy(Sptr->name, "Ann") 로 사용해야 함 !

그림 2-38 데이터 항목 참조 연산 예

3. 구조체 : 구조체 연산

■ 구조체 변수 복사 연산

- 같은 구조체에 있는 구조체 변수들끼리 내용을 한 번에 복사하는 연산
- 예 `struct employee Lee, Kim, team[5];`



(b)

```
Lee = Kim;
Lee = team[2];
team[2] = team[3];
```

그림 2-39 구조체 변수 복사 연산 예

3. 구조체 : 구조체 연산

- 구조체 변수의 주소 구하기 연산
 - 포인터의 주소연산자 사용하여 구조체 변수의 주소 구하거나, 구조체 변수가 배열인 경우에는 배열의 특성에 따라 구조체 배열 변수의 이름에서 주소 구할 수 있음
 - 예

```
struct employee Lee, team[5];
struct employee *Sptr1, *Sptr2;
```

```
Sptr1 = &Lee;
Sptr2 = team;
```

사용 예

```
Sptr1 -> pay = 3000;
Sptr2[0].pay = 3000;
```

그림 2-40 구조체 변수의 주소 구하기 연산 예

4. 재귀호출 : 재귀호출의 개념

- 재귀호출(순환호출)
 - 자기 자신을 호출하여 순환 수행되는 것
 - 함수 실행 특성에 따라 일반적인 호출방식보다 재귀호출방식을 사용하여 함수를 만들면 프로그램의 크기를 줄이고 간단하게 작성 가능
 - 내가 나를 호출하는 것이므로 내 현재 작업을 처리하기 위해 같은 유형의 하위 작업이 필요
 - 하위 작업 : 현재 수행 중인 작업의 하위 단계, 즉 좀 더 작은 단위 작업
 - 전제 문제를 한 번에 해결하기보다 같은 유형의 하위 작업으로 분할하여 작은 문제부터 해결하는 방법이 효율적인 경우 사용
 - 베이스케이스 base case
 - 재귀호출하는 과정 반복하다 보면, 한 번에 해결할 수 있을 정도로 분할된 작업 단위가 충분히 작아지는 단계

4. 재귀호출 : 재귀호출의 개념

- 예)

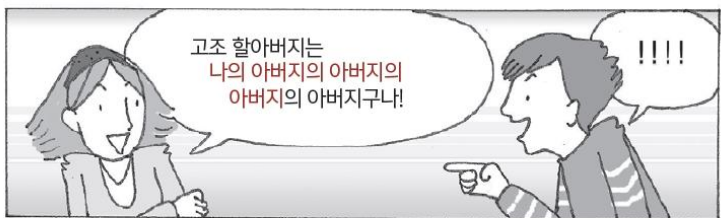
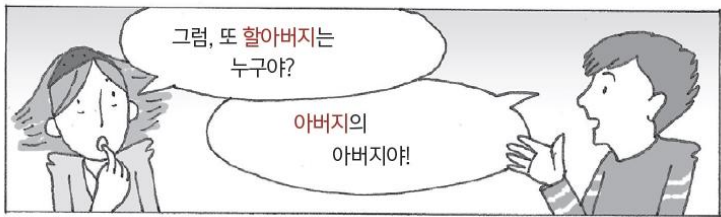
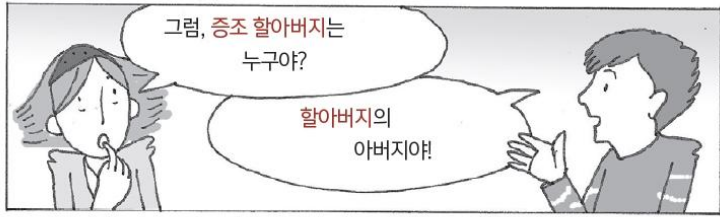
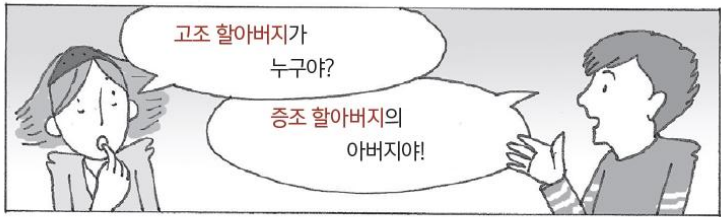


그림 2-41 재귀호출의 개념 예

4. 재귀호출 : 재귀호출의 개념

- 재귀호출 개념의 이해 예)



한 번에 완성할 수 있는 제일 작은 인형
→ 베이스케이스

그림 2-42 재귀호출 개념을 이해하기 위한 마트료시카 인형

4. 재귀호출 : 재귀호출의 예 1

■ 팩토리얼 함수

– n 에 대한 팩토리얼 함수는 1부터 n 까지 모든 자연수를 곱하는 연산

$$n! = n \times (n-1)!$$

$$(n-1)! = (n-1) \times (n-2)!$$

$$(n-2)! = (n-2) \times (n-3)!$$

...

$$2! = 2 \times 1!$$

$$1! = 1 \quad (\text{베이스케이스})$$

그림 2-43 $n!$ 연산

4. 재귀호출 : 재귀호출의 예 1

– 예)

```
long int fact(int n) {
    if (n <= 1)
        return (1);
    else
        return (n * fact(n - 1));
}
```

(a) 재귀호출을 이용한 팩토리얼 함수

그림 2-44 팩토리얼 함수

```
long int fact(int n) {
    int i, f = 1;
    if (n <= 1)
        return (1);
    else {
        for (i = n; i > 0; i--)
            f = f * i;
        return f;
    }
}
```

(b) 반복문을 이용한 팩토리얼 함수

4. 재귀호출 : 재귀호출의 예 1

- factorial 함수에서 $n=4$ 일 때의 실행

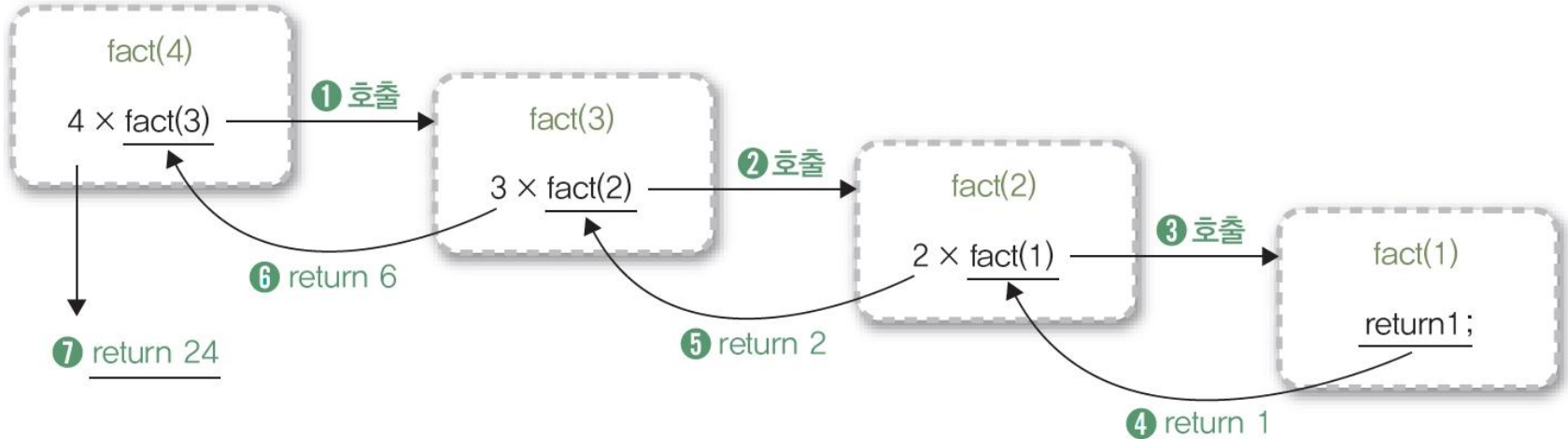


그림 2-45 $n=4$ 일 때 재귀호출과 반환

4. 재귀호출 : 재귀호출의 예 1

- [예제 2-14] : 재귀호출을 이용해 팩토리얼 값 구하기 : [교재 107p](#)

```

정수를 입력하세요 : 4

fact(4) 함수 호출!
fact(3) 함수 호출!
fact(2) 함수 호출!
fact(1) 함수 호출!
fact(1) 값 1 반환!
fact(2) 값 2 반환!
fact(3) 값 6 반환!
fact(4) 값 24 반환!

4의 팩토리얼 값은 24입니다.

```

- 팩토리얼 함수는 단순한 구조라서 재귀호출 방법을 이용하지 않고도 반복문으로 구현 할 수 있다.
- 복잡한 재귀구조 문제는 재귀호출을 이용해야 해결할 수 있다.

4. 재귀호출 : 재귀호출의 예 1

예제 2-14 재귀호출을 이용해 팩토리얼 값 구하기

```

01 #define _CRT_SECURE_NO_WARNINGS
02 #include <stdio.h>
03
04 long int fact(int);
05
06 void main() {
07     int n, result;
08     printf("\n 정수를 입력하세요 : ");
09     scanf("%d", &n);
10     result = fact(n);
11     printf("\n\n %d의 팩토리얼 값은 %ld입니다.\n", n, result);
12     getchar(); getchar();
13 }
14
15 long int fact(int n) {
16     int value;
17     ┌───────────────────────────────────────────────────────────────────────────────────┐
18     │                                                                              │
19     │                                                                              │
20     │                                                                              │
21     │                                                                              │
22     │                                                                              │
23     │                                                                              │
24     │                                                                              │
25     │                                                                              │
26     │                                                                              │
27     └───────────────────────────────────────────────────────────────────────────────────┘
28 }

```

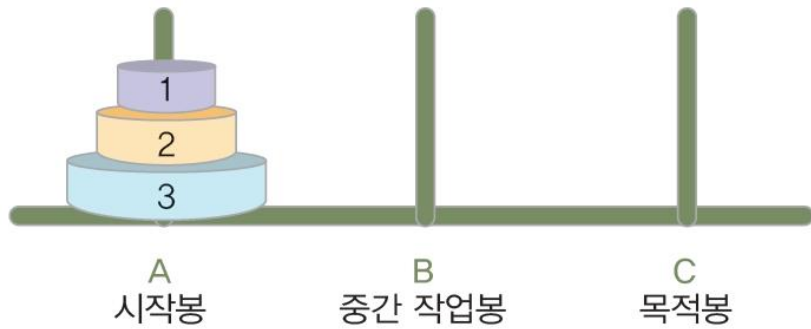
정수를 입력하세요 : 4

fact(4) 함수 호출!
 fact(3) 함수 호출!
 fact(2) 함수 호출!
 fact(1) 함수 호출!
 fact(1) 값 1 반환!
 fact(2) 값 2 반환!
 fact(3) 값 6 반환!
 fact(4) 값 24 반환!

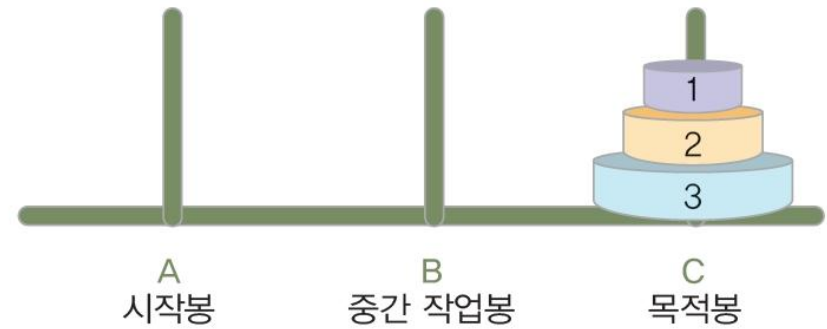
4의 팩토리얼 값은 24입니다.

4. 재귀호출 : 재귀호출의 예 2

- 하노이 탑
 - 복잡한 재귀 구조 문제 해결을 위한 방법
 - 개념 예

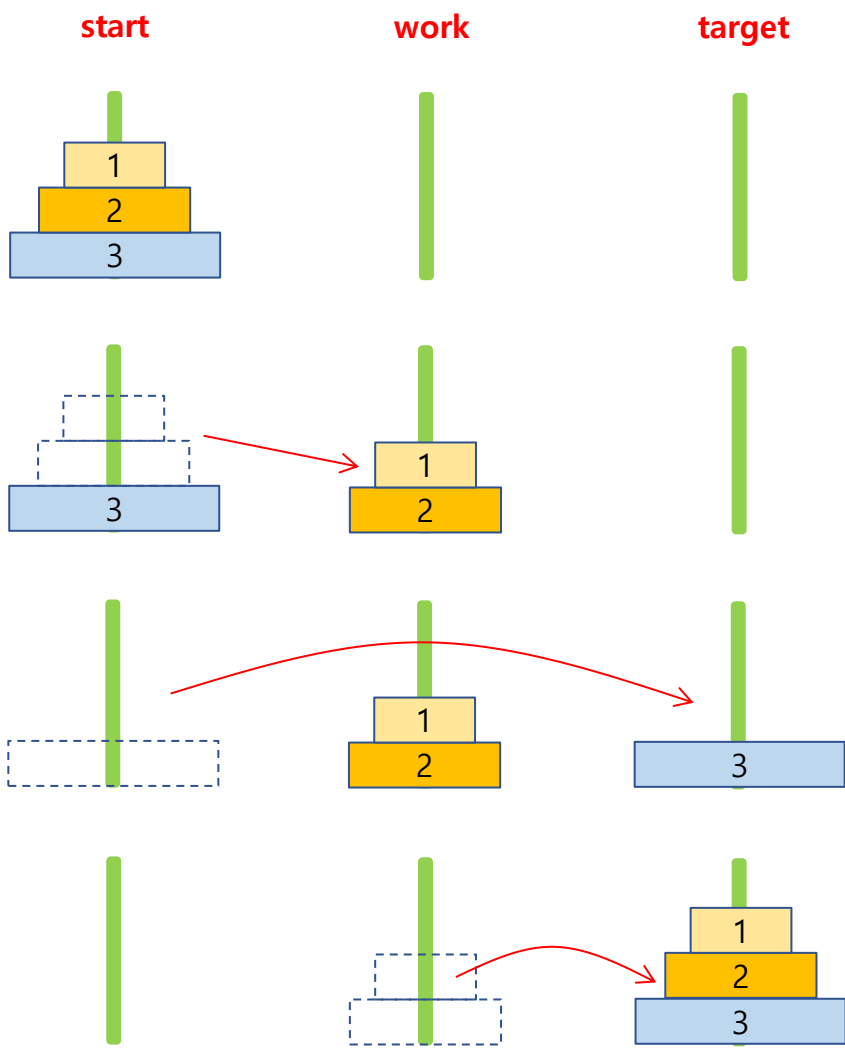


(a) 하노이 탑 퍼즐 시작 상태



(b) 하노이 탑 퍼즐 종료 상태

그림 2-46 하노이 탑 퍼즐의 시작 상태와 종료 상태(원반 개수 $n=3$)



초기상태

어떻게 이동 ?
일단 방법이 있다고 가정하고 생각함.
문제A

start 에서 2 개 원반을
target 를 이용하여
work 로 이동.

start 에서 바닥 원반 1 개를
target 로 이동.

원반이 1개 이므로 쉽다.

work 에서 2 개 원반을
start 를 이용하여
target 로 이동.

문제 A 과 같은 형태의 과정

3개 원반 이동 문제는 2개 원반 이동 문제를 이용하여 풀 수 있다. !!

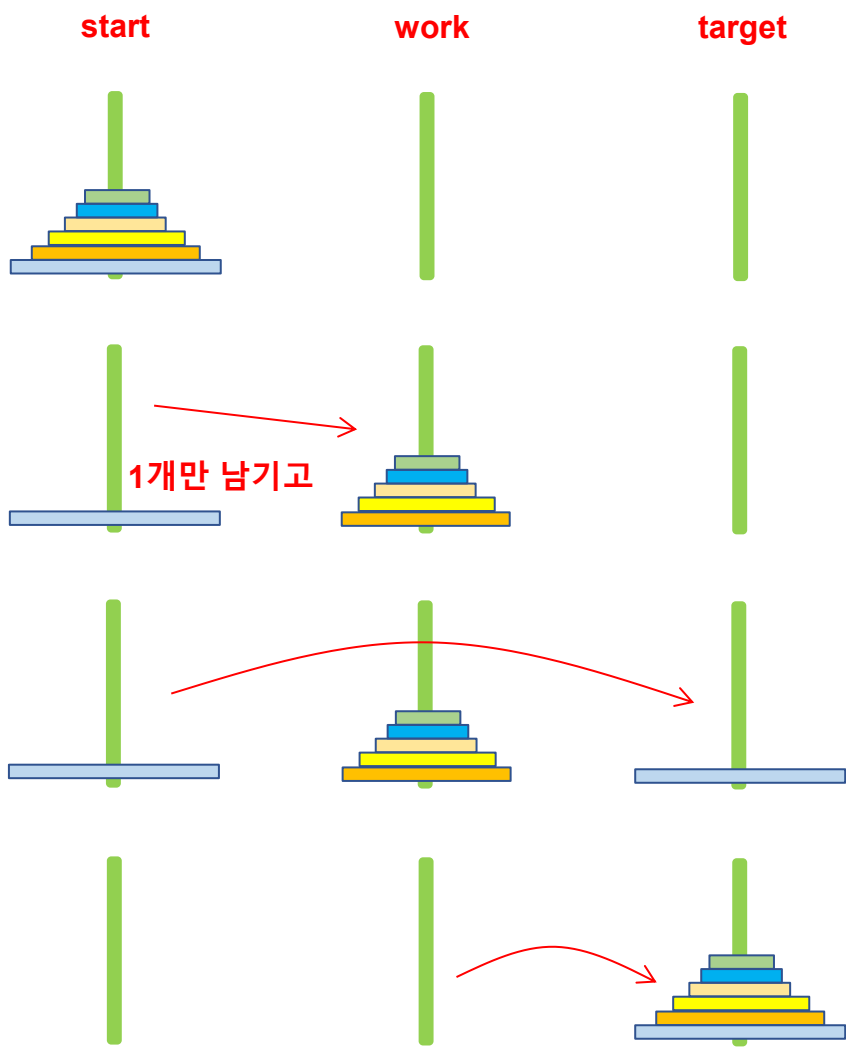
– 문제 A

- start 에서 2 개 원반을 target 를 이용하여 work 로 이동.



- 원래의 문제 (3개 원반) 과 같은 형태의 문제이다.
- 2개 원반 이동 문제는 1개 원반 이동 문제를 이용하여 풀 수 있다.
- 일반화 : N개 원반 문제는 N-1 개 원반 문제를 이용하여 풀 수 있다.
 - ▶ 최종적으로는 1개 원반 이동 문제가 되고 이는 단순하게 해결된다.

N 개 원반 이동



초기상태, N 개

start 에서 N-1 개 원반을
target 를 이용하여
work 로 이동.

문제A

start 에서 바닥 원반 1 개를
target 로 이동.

원반이 1개 이므로 쉽다.

work 에서 N-1 개 원반을
start 를 이용하여
target 로 이동.

문제 A 과 같은 형태의 과정

N 개 원반 이동

- 원하는 작업
 - start 에서 N 개 원반을 work 를 이용하여 target 로 이동
- 함수 매개변수
 - 이동할 원반의 수 : N 개
 - 이동할 원반이 있는 봉 : start
 - 중간과정에 이용할 봉 : work
 - 원반의 최종 목표 봉 : target
- hanoi 함수의 원형
 - `void hanoi ( int n, int start, int work, int target)`
 - 기능 : n 개 원반을 start 에서 work 를 이용하여 target으로 이동
 - 함수 호출이 `hanoi ( n, start, work, target)` 라면 n 개 원반을 start 에서 work 를 거쳐 target 으로 이동.
 - 함수 호출이 `hanoi ( n, work, start, target)` 라면 n 개 원반을 work 에서 start 를 거쳐 target 으로 이동.

4. 재귀호출 : 재귀호출의 예 2

- 하노이 탑 퍼즐의 일반화 개념

1단계(1 ~ 3) : 시작봉^{start}에 있는 원반 1~원반 n-1을, 목적봉^{target}을 이용하여 중간 작업봉^{work}으로 옮긴다.

→ hanoi(n-1, start, target, work)

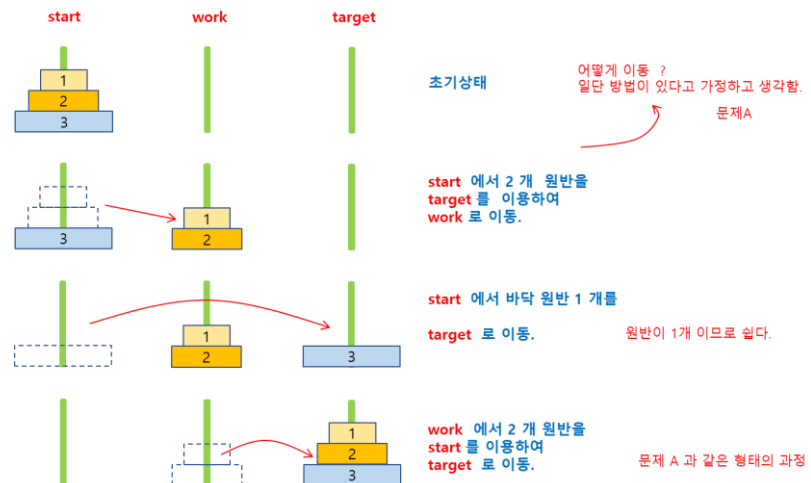
2단계(4) : 시작봉^{start}에 있는 마지막 원반 n을 목적봉^{target}으로 옮긴다.

→ hanoi(1, start, work, target)

3단계(5 ~ 7) : 중간 작업봉^{work}에 있는 원반 1~원반 n-1을, 시작봉^{start}을 이용하여 목적봉^{target}으로 옮긴다.

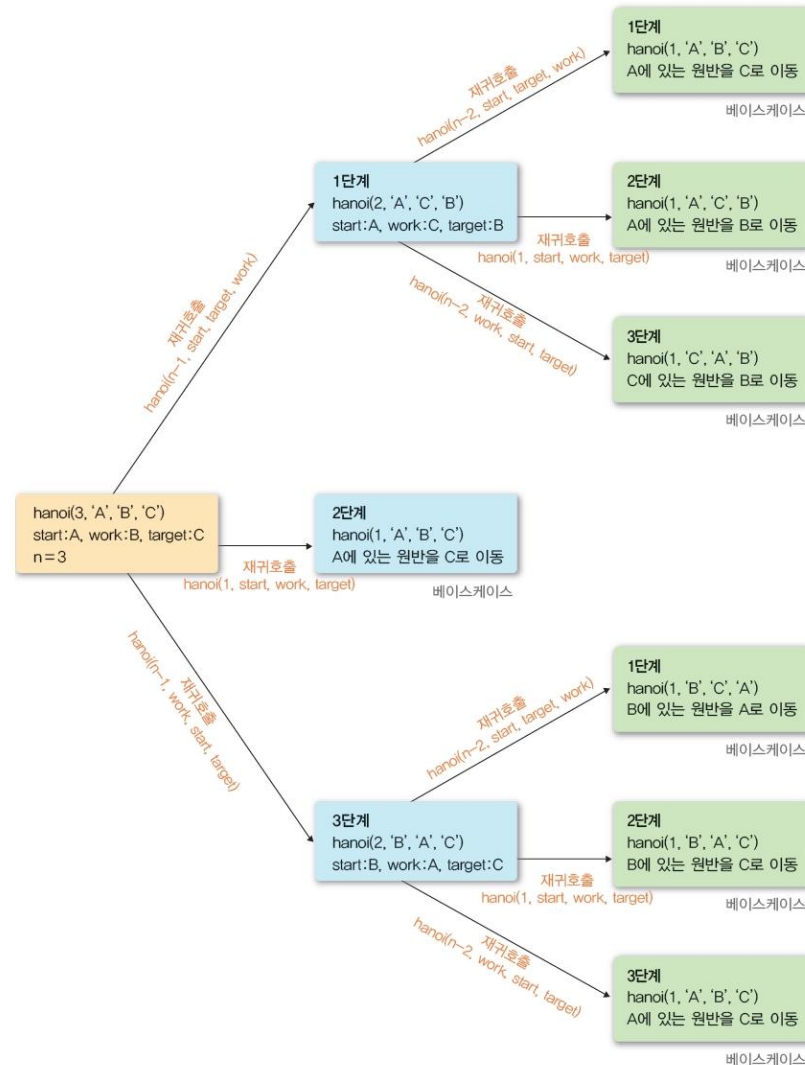
→ hanoi(n-1, work, start, target)

그림 2-47 하노이 탑 작업 과정의 일반화



4. 재귀호출 : 재귀호출의 예 2

- 원반 세 개일 때 하노이 탑 과정을 재귀호출을 이용해 해결하는 과정

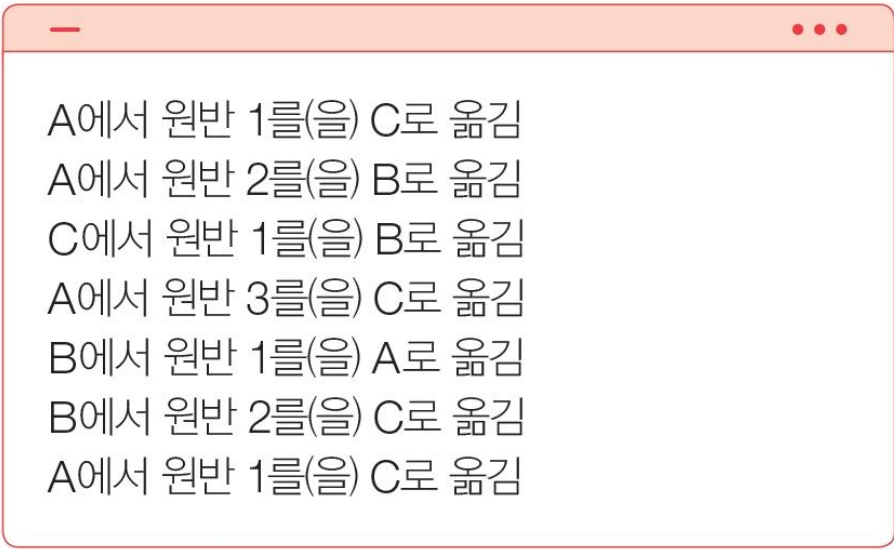


1개 원반 이동 문제 !!

그림 2-48 원반이 세 개일 때 하노이 탑 과정을 재귀호출을 이용해 해결하는 과정

4. 재귀호출 : 재귀호출의 예 2

- [예제 2-15] : 재귀호출을 이용해 하노이 탑 퍼즐 풀기 : [교재 112p](#)



A에서 원반 1를(을) C로 옮김
 A에서 원반 2를(을) B로 옮김
 C에서 원반 1를(을) B로 옮김
 A에서 원반 3를(을) C로 옮김
 B에서 원반 1를(을) A로 옮김
 B에서 원반 2를(을) C로 옮김
 A에서 원반 1를(을) C로 옮김

4. 재귀호출 : 재귀호출의 예 2

```
#include <stdio.h>

void hanoi(int n, int start, int work, int target);

int main(void)
{
    hanoi(3, 'A', 'B', 'C');
    getchar();

    return 0;
}

void hanoi(int n, int start, int work, int target)
{
    // ...
}
```

A에서 원반 1를(을) C로 옮김
 A에서 원반 2를(을) B로 옮김
 C에서 원반 1를(을) B로 옮김
 A에서 원반 3를(을) C로 옮김
 B에서 원반 1를(을) A로 옮김
 B에서 원반 2를(을) C로 옮김
 A에서 원반 1를(을) C로 옮김